Academic Year: 2026-27

Subject: MDM- Wireless & Mobile Communication

Name:	Jay Chandrasen Talekar
Roll No & Branch:	50 / CSEDS
Class/Sem:	TE/V
Experiment No.:	02
Title:	Simulation of basic communication system (Tx–Channel–Rx) using MATLAB / Python / online tools.
Date of Performance:	
Date of Submission:	
Marks:	
Sign of Faculty:	

Aim: Simulation of basic communication system (Tx–Channel–Rx) using MATLAB / Python / online tools.

Objective: To understand the fundamental processes involved in digital communication, including signal generation, transmission through a channel, the effects of noise and distortion, and signal recovery at the receiver. Through simulation, students can observe the behavior and performance of communication systems under different conditions, evaluate transmission reliability, and gain practical knowledge of the principles that govern modern wireless and wired communication networks.

Theory:

A communication system is a set of components that enables the transfer of information from a source to a destination. The primary purpose of a communication system is to transmit messages accurately and efficiently over a communication channel. Communication systems form the foundation of various technologies such as mobile networks, radio broadcasting, television, satellite communication, and the Internet.

A basic communication system consists of three main blocks:

1. Transmitter (Tx)
2. Communication Channel
3. Receiver (Rx)

The transmitted signal travels through a communication channel and is reconstructed at the receiver to recover the original information.

Transmitter (Tx)

The transmitter is responsible for converting the information into a suitable signal for transmission.

Functions of the transmitter include:

- Signal generation
- Encoding
- Modulation
- Signal amplification

The transmitter processes the input message signal and prepares it for efficient transmission through the communication channel.

Communication Channel

The communication channel is the medium through which the signal travels from the transmitter to the receiver.

Types of communication channels:

1. Wired Channel

- Twisted pair cables
- Coaxial cables
- Optical fibers

2. Wireless Channel

- Radio waves
- Microwaves
- Satellite links

During transmission, the signal may be affected by:

- Noise
- Interference
- Attenuation
- Distortion
- Fading

These impairments reduce the quality of the received signal.

Receiver (Rx)

The receiver extracts the original information from the received signal.
Functions of the receiver include:

- Signal reception
- Filtering
- Demodulation
- Decoding
- Signal reconstruction

The quality of the reconstructed signal depends on the amount of noise and distortion introduced by the channel.

Execution Steps

Step 1: Import Required Libraries

Start by importing the required Python libraries.

```
import numpy as np
import matplotlib.pyplot as plt
```

Step 2: Generate the Original Binary Message

A digital communication system starts with information. For this experiment, we generate a random sequence of binary bits.

```
num_bits = 20
```

```
bits = np.random.randint(0, 2, num_bits)

print("Original Bits:")
print(bits)
```

A possible output may be:
Original Bits:
[1 0 1 1 0 0 1 0 1 1 0 1 0 0 1 1 0 1 0 1]

The output changes each time because the bits are randomly generated.

Step 3: Understand BPSK Modulation
BPSK means:
Binary Phase Shift Keying
BPSK uses two signal values to represent two binary states.
We use the following mapping:
Bit 0 → -1
Bit 1 → +1

The conversion can be written as:
BPSK Symbol = 2 × Bit − 1

Therefore:
Bit 0 → 2(0) − 1 = -1
Bit 1 → 2(1) − 1 = +1

We can implement this in Python:

```
symbols = 2 * bits - 1

print("BPSK Symbols:")
print(symbols)
```

Step 4: Visualize the Transmitted Signal
We can plot the transmitted BPSK signal.

```
plt.figure(figsize=(10, 4))

plt.stem(symbols)

plt.title("Transmitted BPSK Signal")
```

```
plt.xlabel("Bit Index")
plt.ylabel("Amplitude")
plt.grid(True)

plt.show()
```

The graph contains two possible amplitude levels:
+1 → Bit 1

-1 → Bit 0

At this stage, the signal is assumed to be transmitted without noise.

Step 5: Model the Communication Channel
In a real communication system, the transmitted signal is affected by noise.
The received signal can be represented as:
Received Signal = Transmitted Signal + Noise

Mathematically:
r = s + n

where:

- r = received signal
- s = transmitted signal
- n = noise

For this simulation, we use Gaussian noise.

Step 6: Add AWGN Noise
We define the noise level using the Signal-to-Noise Ratio (SNR).
snr_db = 5

The SNR is measured in decibels.
A higher SNR means:
Less noise → Better communication

A lower SNR means:
More noise → More errors

Now generate noise:

```
snr_linear = 10 ** (snr_db / 10)

noise_std = np.sqrt(1 / snr_linear)

noise = noise_std * np.random.randn(num_bits)
```

The noisy received signal is:

```
received_signal = symbols + noise
```

Step 7: Plot the Received Signal

```
plt.figure(figsize=(10, 4))

plt.stem(received_signal)

plt.title("Received Signal After AWGN Noise")
plt.xlabel("Bit Index")
plt.ylabel("Amplitude")
plt.grid(True)

plt.show()
```

The received signal will no longer contain only +1 and -1.
For example:
Transmitted Signal:
+1, -1, +1, +1, -1

Received Signal:
+0.85, -1.25, +0.60, +1.10, -0.45

The values are different because noise has been added.
However, the receiver can still determine whether the received value is positive or negative.

Step 8: Perform BPSK Demodulation
The receiver uses a decision rule.
If received signal $\geq 0 \rightarrow$ Bit 1

If received signal $< 0 \rightarrow$ Bit 0

Python implementation:

recovered_bits = (received_signal >= 0).astype(int)

Display the recovered bits:
print("Recovered Bits:")
print(recovered_bits)

Step 9: Compare Original and Recovered Bits
Now compare the transmitted bits and recovered bits.
print("Original Bits:")
print(bits)

print("Recovered Bits:")
print(recovered_bits)

Example:
Original Bits:
1 0 1 1 0 1 0 1 1 0

Recovered Bits:
1 0 1 1 0 1 0 1 1 0

If both sequences are identical, then no bit errors occurred.
If they are different, the noise caused communication errors.

Step 10: Calculate Bit Error Rate (BER)
The Bit Error Rate is calculated using:
BER = Number of Incorrect Bits / Total Number of Transmitted Bits

Python code:
num_errors = np.sum(bits != recovered_bits)

ber = num_errors / num_bits

print("Number of Errors:", num_errors)
print("Bit Error Rate:", ber)

Code -

```
import numpy as np
import matplotlib.pyplot as plt

# Generate Binary Data
num_bits = 20
bits = np.random.randint(0, 2, num_bits)

# Time Axis
bit_duration = 1
samples_per_bit = 100
t = np.arange(0, num_bits * bit_duration,
              bit_duration / samples_per_bit)

# Create NRZ Binary Signal
binary_signal = np.repeat(bits, samples_per_bit)

# Carrier Signal
fc = 5  # Carrier Frequency
carrier = np.sin(2 * np.pi * fc * t)

# BPSK Modulation
bpsk_signal = carrier * (2 * binary_signal - 1)

# Add AWGN Noise
snr_db = 5
signal_power = np.mean(bpsk_signal**2)
noise_power = signal_power / (10**(snr_db/10))

noise = np.sqrt(noise_power) * np.random.randn(len(bpsk_signal))
received_signal = bpsk_signal + noise

# Demodulation
demodulated = received_signal * carrier

# Decision Making
recovered_bits = []
```

```
for i in range(num_bits):
    start = i * samples_per_bit
    end = (i + 1) * samples_per_bit

    if np.sum(demodulated[start:end]) > 0:
        recovered_bits.append(1)
    else:
        recovered_bits.append(0)

recovered_bits = np.array(recovered_bits)

# --------------------------------
# Plot 1: Original Binary Data
# --------------------------------
plt.figure(figsize=(10,4))
plt.step(range(num_bits), bits, where='mid')
plt.title("Original Binary Data")
plt.xlabel("Bit Index")
plt.ylabel("Bit Value")
plt.grid(True)
plt.show()

# --------------------------------
# Plot 2: Sine Wave Carrier
# --------------------------------
plt.figure(figsize=(10,4))
plt.plot(t[:500], carrier[:500])
plt.title("Sine Wave Carrier")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()

# --------------------------------
# Plot 3: Transmitted BPSK Signal
# --------------------------------
plt.figure(figsize=(10,4))
plt.plot(t[:1000], bpsk_signal[:1000])
plt.title("Transmitted BPSK Signal")
```

```
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()

# ------------------------------
# Plot 4: Received Signal with AWGN Noise
# ------------------------------
plt.figure(figsize=(10,4))
plt.plot(t[:1000], received_signal[:1000])
plt.title("Received Signal with AWGN Noise")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()

# ------------------------------
# Plot 5: Demodulated Signal
# ------------------------------
plt.figure(figsize=(10,4))
plt.plot(t[:1000], demodulated[:1000])
plt.title("Demodulated Signal")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()

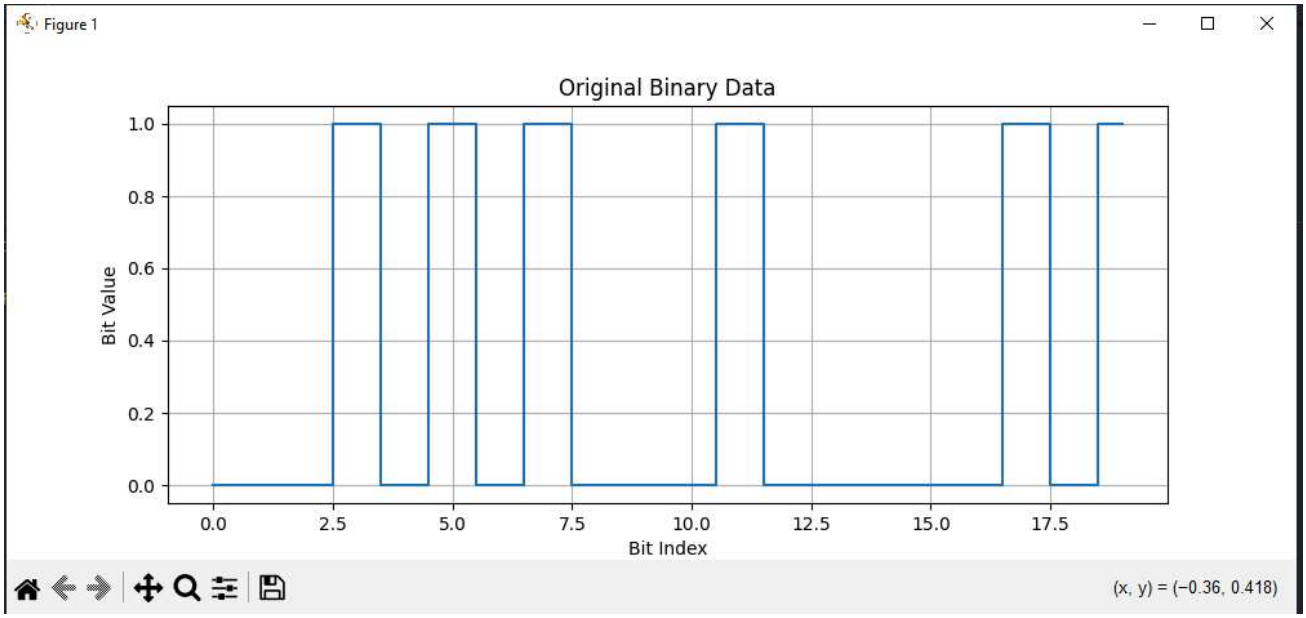
# BER Calculation
errors = np.sum(bits != recovered_bits)
ber = errors / num_bits

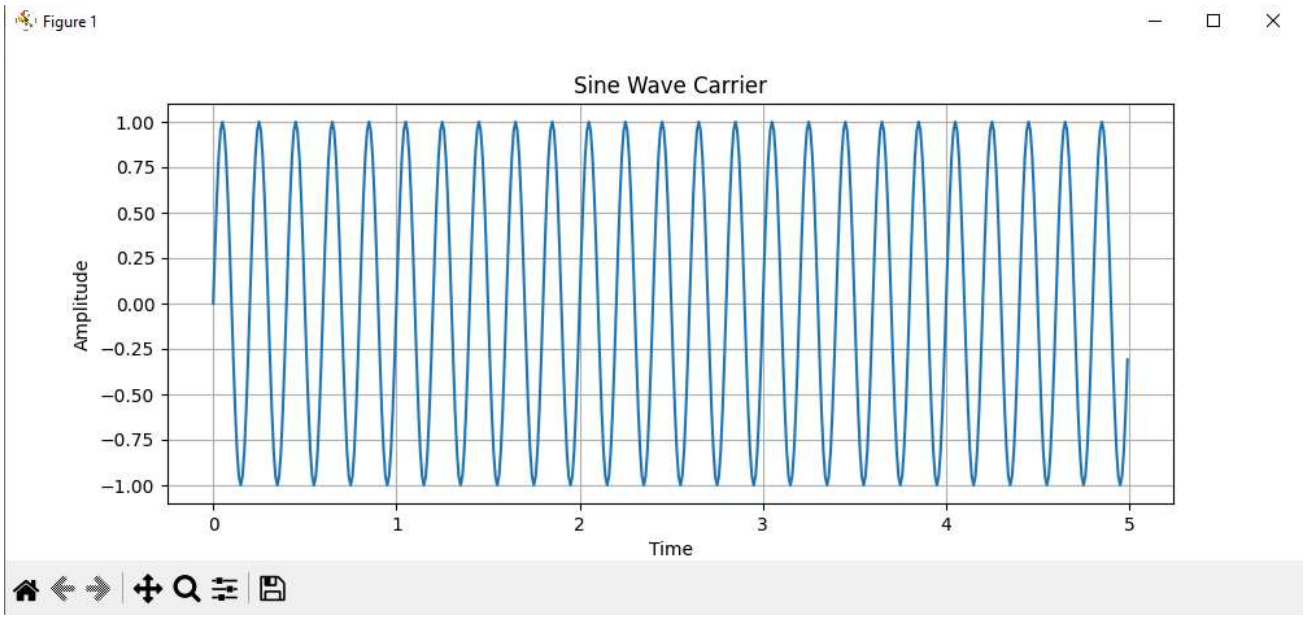
print("Original Bits:")
print(bits)

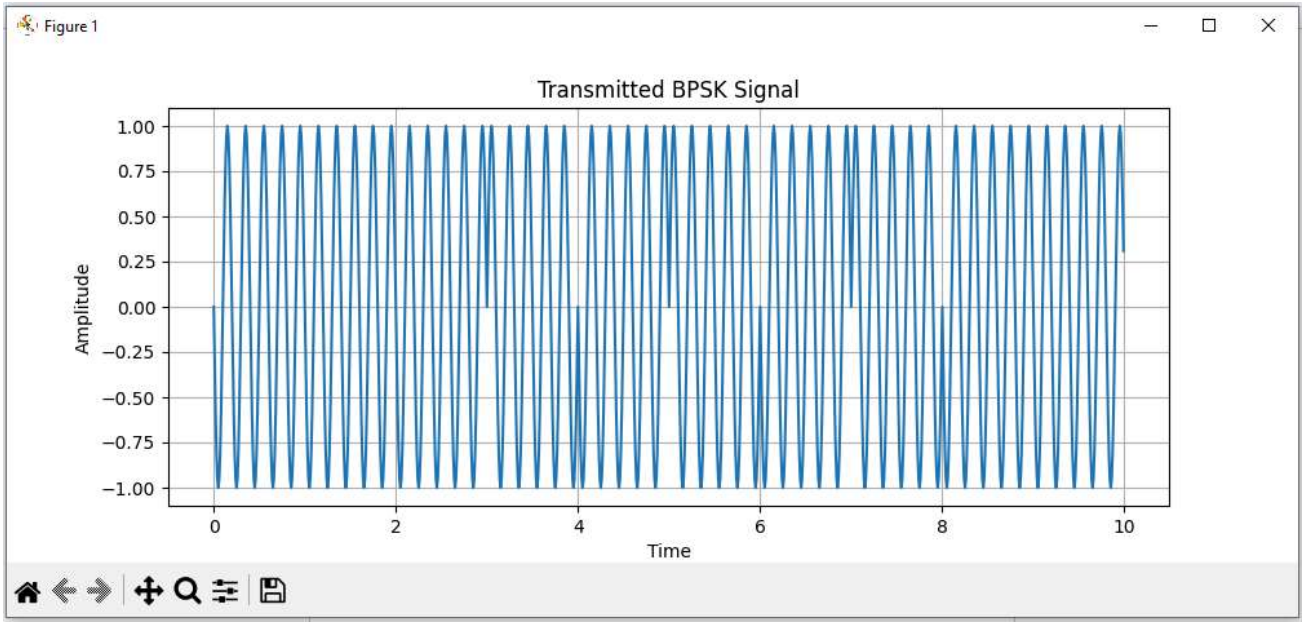
print("\nRecovered Bits:")
print(recovered_bits)

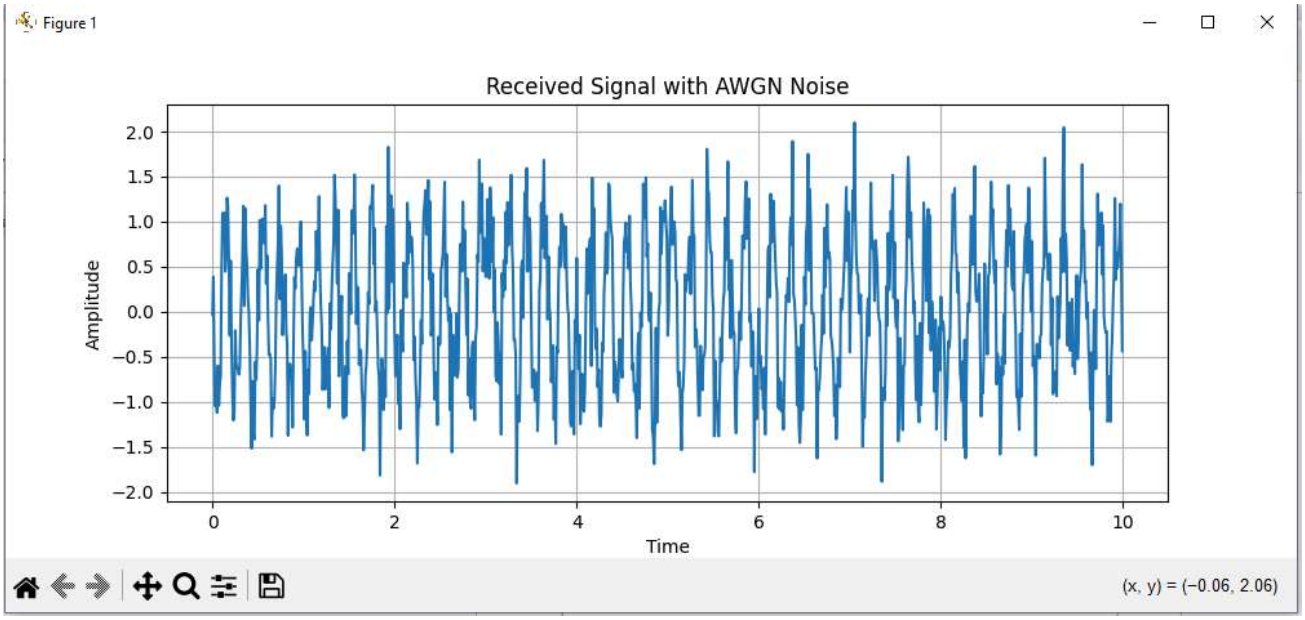
print("\nNumber of Errors:", errors)
print("BER =", ber)
```

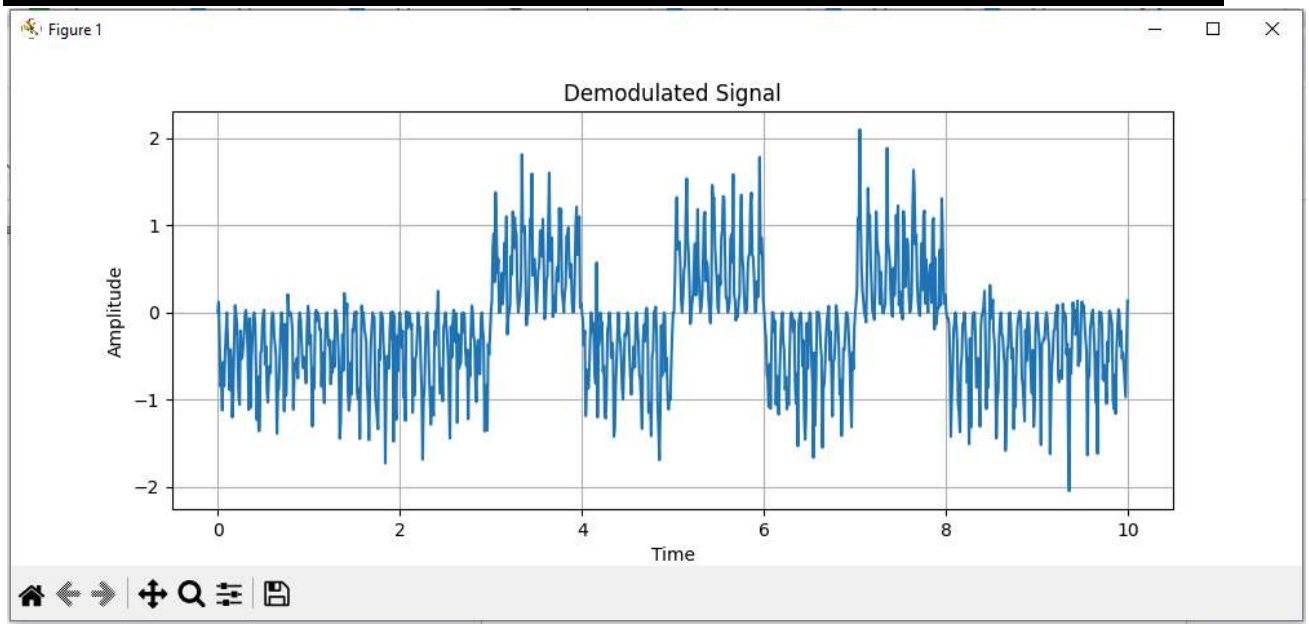
Output -


Figure 1
Original Binary Data
Bit Value
Bit Index
0.0
0.2
0.4
0.6
0.8
1.0
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
(x, y) = (−0.36, 0.418)


Figure 1
Sine Wave Carrier
Amplitude
Time
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0
1
2
3
4
5


Figure 1
Transmitted BPSK Signal
Amplitude
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0
2
4
6
8
10
Time


Figure 1
Received Signal with AWGN Noise
Amplitude
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−2.0
0
2
4
6
8
10
Time
(x, y) = (−0.06, 2.06)




```
PS C:\Users\admin\Desktop\MDM EXP4> python n.py
Original Bits:
[0 0 0 1 0 1 0 1 0 0 0 1 0 0 0 0 0 1 0 1]

Recovered Bits:
[0 0 0 1 0 1 0 1 0 0 0 1 0 0 0 0 0 1 0 1]

Number of Errors: 0
BER = 0.0
```

Conclusion:

A basic communication system consists of a transmitter, communication channel, and receiver working together to transfer information from a source to a destination. Simulation using MATLAB, Python, or online tools helps visualize signal transmission, channel effects, and signal recovery processes. It provides valuable insight into communication system performance, noise effects, and the fundamental principles used in modern communication technologies.